MODULE 5

Model Evaluation and Applications

Introduction to Machine Learning

Complete Study Notes

From first principles to professional practice

Course instructor
Prof. Rajesh Nag

Topics covered

Model validation • Overfitting and underfitting • Cross-validation • Machine learning applications • Case study

Worked examples, diagrams, code, exam tips and a revision toolkit included.

How to Use These Notes

Start with the green *In Plain Words* boxes if the topic is new to you; learn the red *Definition* boxes and the *Worked Examples* for exams; read the purple *Pro Corner* boxes for professional depth.

Box	What it gives you
In Plain Words	Everyday analogy; the idea with no jargon
Definition / Key Idea	Exact wording to write in exams
Worked Example	Step-by-step calculation
Tip	Helpful shortcuts and intuition
Watch Out	Common mistakes and traps
Exam Tip	How questions are usually asked and answered
Pro Corner	Deeper, professional-level details

Suggested study plan: read Chapters 1–3 for the theory (validation, over/underfitting, cross-validation), Chapter 4 for applications, and Chapter 5 for a complete worked case study you can reuse as an exam answer. Chapter 6 holds the revision material.

Contents

How to Use These Notes i

1 Model Validation 1
- 1.1 Why Training Performance Is Not Enough . . . 1
- 1.2 The Validation Workflow . . . 1
- 1.3 Hold-Out Validation . . . 2
- 1.4 Start with a Baseline . . . 2
- 1.5 Choosing the Right Validation Metric . . . 2
- 1.6 Hyperparameter Tuning with Validation Data . . . 2
- 1.7 How Reliable Is a Test Score? . . . 2
- 1.8 Common Validation Mistakes . . . 3
- 1.9 Code . . . 3

2 Overfitting and Underfitting 5
- 2.1 A Picture of the Three Cases . . . 5
- 2.2 Comparing the Diagnoses . . . 6
- 2.3 Bias and Variance . . . 6
- 2.4 Learning Curves: Seeing the Problem . . . 7
- 2.5 How to Fix Underfitting . . . 7
- 2.6 How to Fix Overfitting . . . 8
- 2.7 A Practical Diagnosis Checklist . . . 9

3 Cross-Validation (Basic Idea) 10
- 3.1 Why Not Just One Hold-Out Split? . . . 10
- 3.2 k-Fold Cross-Validation . . . 10
- 3.3 Choosing k . . . 11
- 3.4 Variants . . . 11
- 3.5 Using Cross-Validation to Compare Models . . . 11

3.6 Cross-Validation for Hyperparameter Tuning . . . 11
3.7 Doing It Right: Pipelines . . . 12
3.8 Advantages and Limitations . . . 12

4 Machine Learning Applications 14
4.1 Applications by Domain . . . 14
4.2 Applications by Learning Type . . . 15
4.3 Examples in the Indian Context . . . 15
4.4 From Model to Product: Deployment and MLOps . . . 15
4.5 Choosing the Right Approach: A Quick Guide . . . 16
4.6 Responsible and Ethical Use . . . 16
4.7 Limitations of Machine Learning . . . 16

5 Case Study: Predicting Customer Churn 18
5.1 Step 1: Define the Problem . . . 18
5.2 Step 2: Data . . . 18
5.3 Step 3: Preprocessing . . . 18
5.4 Step 4: Baseline . . . 18
5.5 Step 5: Model Selection with Cross-Validation . . . 19
5.6 Step 6: Check for Overfitting and Underfitting . . . 19
5.7 Step 7: Final Evaluation on the Test Set (Used Once) . . . 19
5.8 Step 8: Choosing the Threshold from Business Costs . . . 20
5.9 Step 9: Deployment and Monitoring . . . 20
5.10 What the Case Study Demonstrated . . . 21
5.11 Code Skeleton for the Whole Case . . . 21

6 Revision Toolkit 23
6.1 One-Page Summary . . . 23
6.2 Formula Sheet . . . 23
6.3 Commonly Confused Pairs . . . 24
6.4 Short-Answer Practice . . . 24
6.5 Long-Answer Outlines . . . 25
6.6 Multiple-Choice Practice . . . 25
6.7 Glossary . . . 26

1 Model Validation

In Plain Words

Passing a driving test in an empty car park proves little; the real test is the busy road. A model that scores well on the data it studied may collapse on new data. **Model validation** is the practice of checking, honestly and in advance, how a model will behave on data it has never seen, before it is trusted with real decisions.

Definition: Model Validation

The process of estimating how well a trained model will **generalise** to new, unseen data, using data that was *not* used to fit the model, so that models and settings can be compared and the best one selected.

1.1 Why Training Performance Is Not Enough

A model can simply *memorise* its training data. A 1-nearest-neighbour classifier, for instance, scores 100% on its own training set (each point is its own nearest neighbour) and may still be poor on new data. The error measured on training data is called the **resubstitution (training) error**, and it is an *optimistic* estimate.

Error	Measured on	Purpose
Training error	Data used to fit the model	Shows whether the model can learn the data at all
Validation error	Held-out data used during development	Choose between models and tune hyperparameters
Test error	Final held-out data, used once	Unbiased estimate of real-world performance
Generalisation error	Expected error on all future data (what we truly care about)	Estimated by the validation / test error

Generalisation gap = validation (or test) error − training error. A large gap warns of overfitting (Chapter 2).

1.2 The Validation Workflow

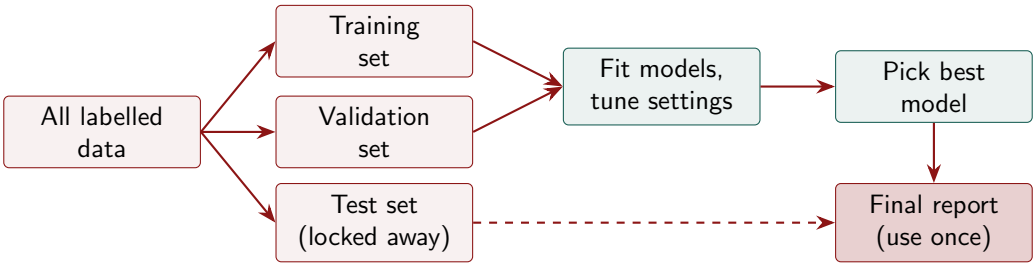


Figure 1. The validation set guides all decisions; the test set is touched only once at the end.

1. **Split** the data (stratified for classification) into training, validation and test sets.
2. **Train** candidate models on the training set.
3. **Validate** each on the validation set; compare scores; tune hyperparameters.
4. **Select** the best model and, optionally, retrain it on training + validation data.

5. **Test** the single chosen model on the test set *once* and report that number.

1.3 Hold-Out Validation

Split once, e.g. 70/15/15 or 60/20/20. It is fast and simple, but the result depends on which samples fell into which set, so with small data it is noisy; this is why cross-validation (Chapter 3) is preferred when data is limited.

1.4 Start with a Baseline

A score means nothing without a yardstick. Always compare against a trivial **baseline**:

- **Classification:** always predict the majority class. If 80% of customers do not churn, the baseline has 80% accuracy, so a model with 82% accuracy is barely better.
- **Regression:** always predict the training mean (this gives $R^2 = 0$).
- A simple model such as logistic regression or a short decision tree is a good second baseline.

1.5 Choosing the Right Validation Metric

Task	Metrics	Remark
Balanced classification	Accuracy	Easy to explain
Imbalanced classification	Precision, recall, F_1, PR-AUC	Accuracy hides rare-class failures
Ranking / thresholds	ROC-AUC	Independent of the cut-off
Regression	MAE, MSE, RMSE, R^2	RMSE punishes large errors; MAE is robust
Clustering (no labels)	Silhouette score	Internal validation

The metric must match the *business cost* of errors (see the case study in Chapter 5).

1.6 Hyperparameter Tuning with Validation Data

Parameters are learned from data (weights); **hyperparameters** are chosen by us (the k in KNN, tree depth, regularisation strength). We pick them using validation performance.

Method	How it works	Remark
Grid search	Try every combination from a fixed grid	Exhaustive; cost grows multiplicatively: $5 \times 4 \times 3 = 60$ runs
Random search	Sample combinations at random	Often finds good settings faster in large spaces
Bayesian optimisation	Uses past results to choose the next trial	Efficient for expensive models

1.7 How Reliable Is a Test Score?

A test score is itself an *estimate* with uncertainty. For accuracy $\hat{p}$ measured on n test samples, the standard error is $\sqrt{\hat{p}(1-\hat{p})/n}$, and an approximate 95% interval is $\hat{p} \pm 1.96\times$ SE.

Worked Example: Confidence interval for accuracy

Accuracy $\hat{p} = 0.85$.
$n = 200$: SE $= \sqrt{\frac{0.85 \times 0.15}{200}} = \sqrt{0.000638} = 0.0253$; interval $0.85 \pm 1.96(0.0253) = 0.85 \pm 0.05$, i.e. roughly **0.80** to **0.90**.
$n = 2000$: SE $= 0.0080$; interval $\approx 0.85 \pm 0.016$, i.e. 0.834 to 0.866.
A **bigger test set gives a tighter estimate**. A claim that model A (0.85) beats model B (0.84) on 200 samples is not trustworthy.

1.8 Common Validation Mistakes

- **Data leakage:** fitting scalers, imputers or feature selectors on all the data before splitting; duplicates in both train and test; features that reveal the answer.
- **Test-set reuse:** tuning until the test score looks good turns it into a second validation set.
- **Wrong split for the problem:** random shuffling of time-ordered data (use past $\rightarrow$ future); splitting rows of the same patient or user across sets (use group splits).
- **Unrepresentative data:** validation data that does not match what the model will see in production.
- **Ignoring variance:** drawing conclusions from one split or a tiny difference.

Key Idea

Golden rules of validation: (1) split before any learning step; (2) all decisions use training/validation data only; (3) the test set is used once; (4) always compare with a baseline; (5) choose metrics that reflect real costs; (6) report uncertainty, not just a single number.

1.9 Code

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import f1_score

# 60 / 20 / 20 split
X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
X_tr, X_val, y_tr, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)

base = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)   # baseline
print("Baseline F1:", f1_score(y_val, base.predict(X_val), zero_division=0))

best = None
for k in [1, 3, 5, 7, 9, 11]:                        # tune k on VALIDATION data
    m = KNeighborsClassifier(n_neighbors=k).fit(X_tr, y_tr)
    s = f1_score(y_val, m.predict(X_val))
    if best is None or s > best[0]:
        best = (s, k)
print("Best k:", best[1], "validation F1:", best[0])

final = KNeighborsClassifier(n_neighbors=best[1]).fit(X_tmp, y_tmp)
print("Test F1 (reported once):", f1_score(y_test, final.predict(X_test)))
```

Exam Tip

Likely questions: "What is model validation and why is it needed?", "Differentiate training, validation and test sets", "What is a baseline model?" Write the three-set table, draw the workflow figure, and mention data leakage and test-set reuse as pitfalls.

Pro Corner (for professionals)

Selection bias and optimism. Picking the best of many models on the same validation set inflates its score (the *winner's curse*), because the maximum of noisy estimates is biased upward. The remedy is a fresh test set or *nested cross-validation*. By Hoeffding's inequality, for a bounded loss in $[0, 1]$ the deviation of an empirical mean from the true risk exceeds ϵ with probability at most $2e^{-2n\epsilon^2}$, which is why test-set size controls certainty. In production, **offline** validation is complemented by **online** evaluation (shadow deployment, A/B tests) because offline metrics may not capture user behaviour, feedback loops, or distribution shift.

2 Overfitting and Underfitting

In Plain Words

Three students prepare for an exam. One barely studies and fails everything (**underfitting**). One memorises last year's paper word for word, scores full marks on those questions and collapses on new ones (**overfitting**). The third understands the concepts and handles fresh questions (**good fit**). Machine learning aims to produce the third student.

Definition: Underfitting

The model is **too simple** to capture the real pattern in the data. It performs poorly on *both* training and new data (high bias).

Definition: Overfitting

The model fits the training data **too closely, including noise and accidents**, so it performs very well on training data but poorly on new data (high variance).

2.1 A Picture of the Three Cases

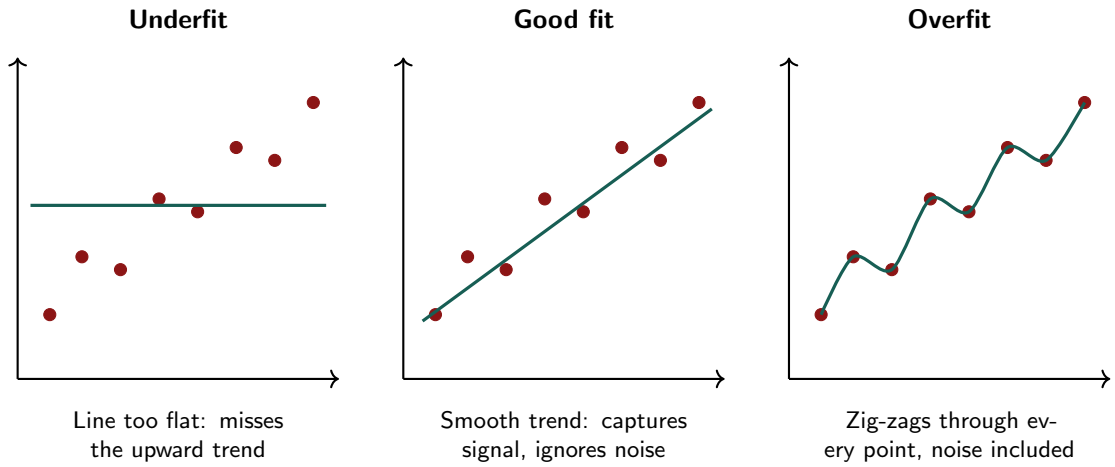


Figure 2. The same eight data points fitted by a model that is too simple, just right, and too complex.

2.2 Comparing the Diagnoses

	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Validation / test error	High	Low, close to training error	Much higher than training error
Generalisation gap	Small	Small	**Large**
Bias / variance	High bias, low variance	Balanced	Low bias, high variance
Model complexity	Too low	Appropriate	Too high
Typical cause	Too few features, too simple model, too much regularisation, too little training	Right model and data	Too complex model, too little data, noisy data, too long training

Worked Example: Polynomial regression (illustrative numbers)

Fitting polynomials of different degree to the same data set gave:

Degree	Train MSE	Test MSE	Diagnosis
1	25.0	27.0	Underfitting: both errors high
3	4.0	5.0	Good fit: both low and close
9	0.2	40.0	Overfitting: train near zero, test huge

Training error alone always falls as complexity rises (0.2 looks "best"). Only the **test/validation error** reveals that degree 3 is the right choice.

2.3 Bias and Variance

- **Bias:** error from wrong or overly simple assumptions. A straight line fitted to a curve is biased no matter how much data you give it.
- **Variance:** error from sensitivity to the particular training sample. A very flexible model changes wildly when the training data changes slightly.
- **Irreducible noise:** randomness no model can remove.

$$\text{Expected squared error} = \text{Bias}^2 + \text{Variance} + \text{Noise}.$$

Worked Example: Bias vs. variance with numbers

The true value at some input is 10. We train each model on four different samples and record its prediction at that input.
Model A (simple): 12, 14, 13, 13. Mean $= 13$, so bias $= 13 - 10 = 3$ and bias$^2 = 9$. Variance $= \frac{(12-13)^2+(14-13)^2+0+0}{4} = 0.5$. Mean squared error $= 9 + 0.5 = 9.5$.
Model B (flexible): 8, 14, 6, 12. Mean $= 10$, so bias $= 0$. Variance $= \frac{4+16+16+4}{4} = 10$. Mean squared error $= 0 + 10 = 10$.
Model A is *consistently wrong* (high bias, low variance: underfitting-type error). Model B is *right on average but erratic* (low bias, high variance: overfitting-type error). Both have a similar total error, for opposite reasons, and need opposite cures.

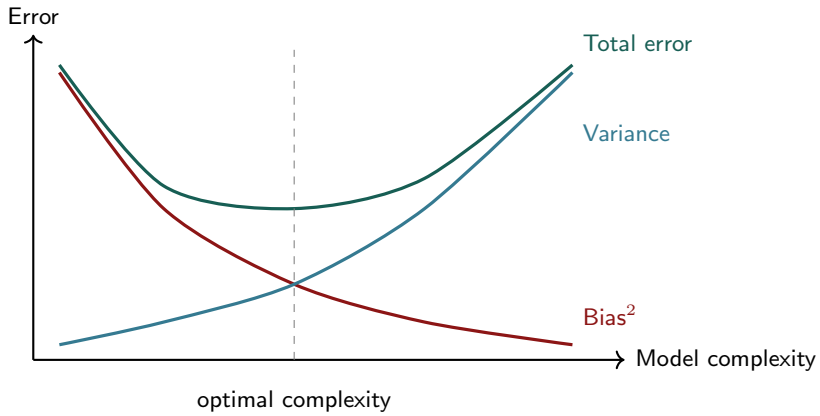
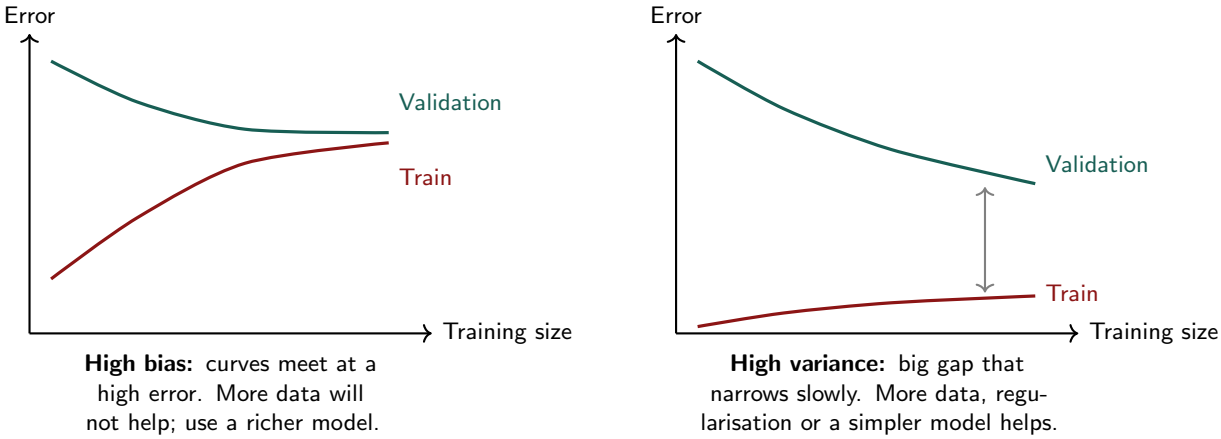


Figure 3. The bias–variance trade-off: total error is lowest at a balanced complexity.

2.4 Learning Curves: Seeing the Problem

A **learning curve** plots training and validation scores against the *training-set size*.



High bias: curves meet at a high error. More data will not help; use a richer model.

High variance: big gap that narrows slowly. More data, regularisation or a simpler model helps.

Figure 4. Learning curves diagnose underfitting (left) and overfitting (right).

2.5 How to Fix Underfitting

1. Use a **more expressive model** (add polynomial terms, deeper tree, more neurons).
2. Add or engineer **more informative features**.
3. **Reduce regularisation** (smaller λ).
4. **Train longer**, or improve the optimiser and learning rate.
5. Remove noise or errors in the data that prevent learning.

2.6 How to Fix Overfitting

Remedy	How it helps	Example
Get more data	Harder to memorise a large, varied sample	Collect more labelled examples; data augmentation (flip, crop images)
Simplify the model	Fewer ways to chase noise	Lower polynomial degree; limit tree depth; larger k in KNN
Regularisation	Penalises large weights	L2: $J = \text{MSE} + \lambda \sum w^2$; L1 for sparse weights
Feature selection	Removes noisy, irrelevant inputs	Filter / wrapper / embedded methods
Early stopping	Stop training when validation error starts rising	Neural networks, gradient boosting
Pruning	Cut back branches that fit noise	Decision trees (pre- or post-pruning)
Dropout	Randomly switch off neurons during training	Neural networks
Ensembling (bagging)	Averaging many models cancels their individual noise	Random forest
Cross-validation	Reveals instability before deployment	k-fold CV for model selection

Worked Example: Regularisation changes the loss

A linear model has weights $w = (3, -2)$ and data-fit error MSE $= 2.0$. With ridge penalty $\lambda = 0.5$:
$J = \text{MSE} + \lambda \sum w_j^2 = 2.0 + 0.5(9 + 4) = 2.0 + 6.5 = 8.5$.
To reduce J, the optimiser must shrink large weights, which gives a smoother, less overfitted model. A bigger λ simplifies the model more (risking underfitting); $\lambda = 0$ means no regularisation.

Worked Example: Early stopping

Loss recorded after each training epoch:

Epoch	1	2	3	4	5	6	7	8
Training loss	0.90	0.60	0.42	0.31	0.22	0.15	0.10	0.06
Validation loss	0.95	0.65	0.50	0.44	**0.43**	0.47	0.54	0.63

Training loss keeps falling, but validation loss is lowest at **epoch 5** and then climbs: the model has begun to memorise. Stop at epoch 5 (or keep the weights from that epoch).

2.7 A Practical Diagnosis Checklist

Observation	Likely problem	First action
Train and validation both poor	Underfitting	Richer model or features
Train excellent, validation poor	Overfitting	Regularise, simplify, add data
Both excellent, test poor	Validation leakage or unrepresentative split	Audit the pipeline and data splits
Validation fluctuates wildly between folds	High variance, small data	Cross-validation, simpler model
Perfect score on first try	Probable leakage	Check for target leakage and duplicates
Good offline, poor in production	Distribution shift or training–serving skew	Monitor and retrain on fresh data

Watch Out

Do not "fix" overfitting by tuning until the *test* score improves: that overfits the test set. Make every adjustment using the validation set or cross-validation.

Exam Tip

A near-certain question: "Explain overfitting and underfitting with diagrams. How can they be avoided?" Draw the three-panel figure, give the comparison table, then list remedies (more data, regularisation, early stopping, pruning, cross-validation, feature selection). Add the bias–variance sentence for extra marks.

Pro Corner (for professionals)

Beyond the classical picture. The U-shaped test-error curve describes classical models, but very large neural networks can show *double descent*: test error peaks near the point where the model just fits the training data and then *decreases* again as capacity grows further, helped by implicit regularisation in gradient descent. Capacity can be reasoned about with the VC dimension: with probability $1-\delta$, true risk $\leq$ empirical risk $+\sqrt{\frac{d_{VC}(\ln(2n/d_{VC})+1)+\ln(4/\delta)}{n}}$, so the gap shrinks as n grows and widens with model capacity. In practice, combine regularisation (weight decay, dropout, data augmentation), proper validation, and ensembling; bagging reduces variance, boosting reduces bias.

3 Cross-Validation (Basic Idea)

In Plain Words

Suppose you have only one set of 100 practice questions. If you use 80 to study and 20 to test yourself, your score depends on which 20 happened to be in the test. A fairer plan: split the questions into 5 groups of 20 and take *five* mini-tests, each time studying on four groups and testing on the remaining one. Averaging the five scores gives a much steadier picture. That is cross-validation.

Definition: Cross-Validation (CV)

A resampling technique that estimates model performance by repeatedly **training on part of the data and validating on the remaining part**, rotating which part is held out, and averaging the results.

3.1 Why Not Just One Hold-Out Split?

- With small data, a single split wastes samples and gives a **noisy** estimate: a lucky or unlucky split changes the conclusion.
- Cross-validation uses **every sample for both training and validation**, in different rounds.
- The spread of fold scores also shows how **stable** the model is.

3.2 k-Fold Cross-Validation

1. Shuffle the data and divide it into k equal-sized **folds**.
2. For $i = 1, \ldots, k$: train on the other $k-1$ folds; validate on fold i; record the score s_i.
3. Report the mean $\bar{s} = \frac{1}{k}\sum s_i$ and the standard deviation across folds.

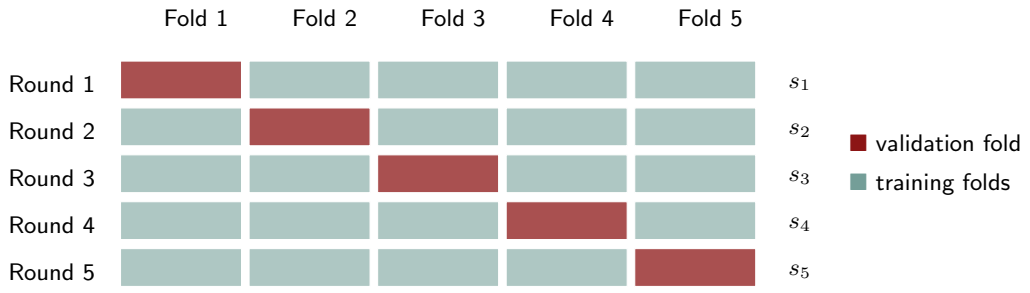


Figure 5. 5-fold cross-validation. Each fold is the validation set exactly once; the five scores $s_1, \ldots, s_5$ are averaged.

Worked Example: Reading 5-fold results

100 samples, $k = 5$: each fold holds 20 samples, so every round trains on 80 and validates on 20.
Accuracies: 0.82, 0.85, 0.80, 0.88, 0.85.
Mean $= \frac{0.82+0.85+0.80+0.88+0.85}{5} = \frac{4.20}{5} = 0.84$. Deviations: -0.02, 0.01, -0.04, 0.04, 0.01; squares sum to 0.0038; sample standard deviation $= \sqrt{0.0038/4} \approx 0.031$.
Report: **accuracy** $= 0.84 \pm 0.03$.

3.3 Choosing k

k	Effect	Use
2–3	Training sets are small, estimate is pessimistic and cheap	Very large or expensive models
5 or 10	Good balance of bias, variance and cost	**Standard default**
n **(LOOCV)**	Almost unbiased, highest variance of estimate and cost	Tiny datasets

Cost: k-fold CV trains the model k times. For $n = 50$, leave-one-out needs 50 trainings; 5-fold needs only 5.

3.4 Variants

Variant	Idea	When to use
Stratified k-fold	Each fold keeps the overall class proportions	Classification, especially imbalanced data
Leave-one-out (LOOCV)	$k = n$: validate on one sample each time	Very small datasets
Repeated k-fold	Repeat k-fold with different shuffles	Lower-variance estimate
Group k-fold	All rows of one group (patient, user) stay in the same fold	Repeated measurements per entity
Time-series split	Train on the past, validate on the following period; never shuffle	Forecasting and any time-ordered data
Leave-p-out	Hold out every possible set of p samples	Rare; computationally heavy

3.5 Using Cross-Validation to Compare Models

Worked Example: Same mean, different stability

5-fold accuracies for two models:

Model	F1	F2	F3	F4	F5	Mean	Std
A	0.84	0.86	0.82	0.85	0.88	0.85	0.022
B	0.90	0.75	0.92	0.78	0.90	0.85	0.079

Means are identical (0.85). Model B swings between 0.75 and 0.92 depending on the data it sees (high variance), while Model A is steady. **Prefer Model A:** its performance is more predictable.

3.6 Cross-Validation for Hyperparameter Tuning

Run CV for every candidate setting (e.g. $k \in \{1, 3, 5, 7, 9\}$ in KNN) and keep the setting with the best mean score. Finally, retrain on *all training data* with that setting and evaluate once on the held-out **test set**. Cross-validation does *not* replace the test set; it replaces the single validation split.

3.7 Doing It Right: Pipelines

Every preprocessing step (imputation, scaling, feature selection, resampling) must be learned **inside each training fold only**. If you scale the whole dataset first, information from the validation fold leaks into training and the CV score becomes over-optimistic. Wrap the steps and the model in a single pipeline.

```
from sklearn.model_selection import (cross_val_score, cross_validate,
                                     StratifiedKFold, GridSearchCV,
                                     TimeSeriesSplit)
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())  # scaler is refit per fold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="f1")
print("F1 per fold:", scores)
print("Mean +/- std: %.3f +/- %.3f" % (scores.mean(), scores.std(ddof=1)))

grid = GridSearchCV(pipe, {"kneighborsclassifier__n_neighbors": [3, 5, 7, 9]},
                    cv=cv, scoring="f1")
grid.fit(X_train, y_train)
print(grid.best_params_, "CV F1 =", grid.best_score_)
print("Test F1:", grid.score(X_test, y_test))        # used once, at the end

# time-ordered data
tscv = TimeSeriesSplit(n_splits=5)
```

3.8 Advantages and Limitations

Advantages	Limitations
Uses all data for training and validation	Costs k times the training effort
More reliable estimate than one split	Not suitable for time-ordered data unless a time-aware split is used
Shows variability (standard deviation)	Leaks if preprocessing is done outside the folds
Works well for small datasets	Still needs a final untouched test set

Watch Out

Never shuffle time series before cross-validating, never split one patient's records across train and validation folds, and never let the test set take part in cross-validation.

Exam Tip

Expect: "Explain k-fold cross-validation with a diagram" and "Why is cross-validation preferred over a simple train–test split?". Draw the 5×5 grid, state the steps, show the *mean of fold scores*, and mention stratification and leakage. The 0.84 ± 0.03 example above can be reused as a numerical.

Pro Corner (for professionals)

Bias and variance of the CV estimate. k-fold CV estimates performance for a model trained on $\frac{k-1}{k}n$ samples, slightly pessimistic for small k; LOOCV is nearly unbiased but its fold models are highly correlated, so the averaged estimate has high variance. There is no unbiased estimator of the CV variance, so the naive standard error $s/\sqrt{k}$ is optimistic; use repeated CV or corrected resampled t-tests when comparing models. **Nested CV** (inner loop to tune, outer loop to estimate) avoids the optimistic bias of tuning and evaluating on the same folds. Use *blocked* or *purged* CV with embargo gaps for autocorrelated data such as finance.

4 Machine Learning Applications

In Plain Words

ML quietly runs much of daily life: the video that appears next on your phone, the bank alert about a suspicious payment, the map that predicts traffic, the translation of a menu in another language. Behind each is the same pipeline you have studied: data, features, a model, and honest evaluation.

4.1 Applications by Domain

Domain	Application	Typical algorithms	Key metric
Healthcare	Disease detection from scans; risk prediction; drug discovery; patient triage	CNNs, logistic regression, random forest	Recall (sensitivity), AUC
Finance	Fraud detection; credit scoring; algorithmic trading; loan default	Gradient boosting, logistic regression, anomaly detection	Precision, recall, PR-AUC, cost
Retail and e-commerce	Recommendations; demand forecasting; dynamic pricing; churn prediction	Collaborative filtering, time-series models, boosting	Click-through, RMSE, F_1
Transport	Self-driving; ETA and route prediction; traffic control	Deep learning, reinforcement learning	Safety metrics, MAE
Agriculture	Crop-yield prediction; leaf-disease detection; irrigation advice; price forecasting	CNNs, regression, time series	Accuracy, RMSE
Education	Personalised learning; dropout prediction; automatic grading	Classification, clustering	Recall for at-risk students
Manufacturing	Predictive maintenance; visual defect detection	Anomaly detection, CNNs	Recall, false-alarm rate
Language (NLP)	Translation; chatbots; sentiment analysis; summarisation	Transformers, Naïve Bayes, logistic regression	BLEU, F_1, accuracy
Computer vision	Face recognition; OCR; medical imaging; quality inspection	CNNs, vision transformers	mAP, accuracy
Cybersecurity	Intrusion, malware and phishing detection	Random forest, anomaly detection	Recall, false-positive rate
Energy and cities	Load forecasting; smart grids; waste and traffic management	Time-series models, RL	MAPE, RMSE
Government and public services	Fraud detection in benefits; crop and flood forecasting; document digitisation	Classification, forecasting	Domain-specific

4.2 Applications by Learning Type

Type	Application examples	Task
Supervised	Spam filter, loan approval, house price, disease diagnosis	Classification, regression
Unsupervised	Customer segmentation, topic discovery, anomaly detection, market-basket analysis	Clustering, association, dimensionality reduction
Semi- / self-supervised	Language models, medical imaging with few labels	Representation learning
Reinforcement	Game playing, robotics, traffic-signal control, ad bidding	Sequential decision making

4.3 Examples in the Indian Context

- **Agriculture:** crop-yield and pest prediction from weather and satellite data; plant-disease detection from phone photos of leaves.
- **Digital payments and banking:** real-time fraud detection on card and UPI transactions; credit scoring for people without a long credit history.
- **Languages:** speech recognition and translation across Indian languages, making services accessible beyond English.
- **Healthcare access:** screening support for eye disease, tuberculosis and cancer where specialists are scarce.
- **Transport and cities:** traffic prediction, public-transport demand forecasting, flood and monsoon forecasting.

4.4 From Model to Product: Deployment and MLOps

A model that scores well in a notebook is only the start. Real systems need:

1. **Deployment:** serve predictions through an API, a mobile app, or scheduled batch jobs.
2. **Monitoring:** track input data, predictions and (when labels arrive) accuracy.
3. **Drift detection:** *data drift* (input distribution changes), *concept drift* (the relationship between inputs and output changes), *label drift*.
4. **Retraining:** refresh the model on recent data on a schedule or when performance drops.
5. **Versioning and reproducibility:** version data, code and models; keep experiment records.
6. **Online evaluation:** A/B tests compare the new model with the old on real users before full rollout.

Worked Example: Data drift in a loan model

A credit model was trained when most applicants had salaried jobs. After a few years many applicants are gig workers with irregular income. The inputs now differ from the training data, so accuracy quietly falls even though the code is unchanged. Monitoring feature distributions would raise an alert; retraining on recent data would fix it.

4.5 Choosing the Right Approach: A Quick Guide

If the problem is...	Start with...
Predict a category from labelled data	Logistic regression, KNN, Naïve Bayes, decision tree; compare with cross-validation
Predict a number	Linear regression, regularised regression, tree-based regression
Group similar items with no labels	k-Means, hierarchical clustering
Find unusual items	Isolation forest, one-class methods, statistical thresholds
Text with little labelled data	Naïve Bayes or logistic regression on TF-IDF, then pre-trained language models
Images	Convolutional networks, transfer learning
Sequential decisions with rewards	Reinforcement learning

4.6 Responsible and Ethical Use

Concern	Risk	Safeguard
Bias and fairness	Models repeat unfair patterns in historical data	Audit data and outcomes across groups; fairness metrics
Privacy	Sensitive personal data exposed or misused	Consent, anonymisation, data minimisation, secure storage
Transparency	People cannot understand decisions that affect them	Interpretable models, explanations, documentation
Accountability	Unclear who is responsible for automated decisions	Human review, audit trails, clear ownership
Safety and robustness	Failures or adversarial inputs cause harm	Testing, monitoring, fallbacks, human oversight
Environmental cost	Large models consume much energy	Efficient models, reuse of pre-trained models

4.7 Limitations of Machine Learning

- Needs sufficient, representative, good-quality data; cannot learn what the data does not show.
- Finds correlations, not causes; predictions can be right for the wrong reasons.
- Can fail on situations unlike the training data (distribution shift) and may be confidently wrong.
- Complex models are hard to explain, which is a problem in medicine, law and finance.
- Needs ongoing maintenance after deployment.

Exam Tip

For "Discuss applications of ML", choose five domains and for each give: *what is predicted*, *the learning type*, *an algorithm*, and *how success is measured*. Finish with one paragraph on deployment, drift and ethics; this shows understanding beyond a simple list.

Pro Corner (for professionals)

Offline vs. business metrics. A model optimised for AUC may not improve the business goal (revenue, safety, retention). Define an *overall evaluation criterion*, convert it into costs where possible (expected cost $= c_{FP}\,FP + c_{FN}\,FN$), and validate with controlled online experiments, watching for *feedback loops* (a recommender changes the data it later learns from) and *training–serving skew* (features computed differently offline and online). Regulated settings add model documentation (model cards, datasheets), explainability (SHAP, counterfactuals) and monitoring for fairness drift.

5 Case Study: Predicting Customer Churn

In Plain Words

A mobile-network company loses customers every month, and winning a new customer costs far more than keeping an old one. If the company could *predict which customers are about to leave*, it could offer them a discount or call them first. This chapter walks through that whole project, step by step, using every idea from the course: data preparation, a baseline, several classifiers, cross-validation, over/underfitting checks, the confusion matrix, and a business decision.

Note: the numbers below are *illustrative*, chosen so the arithmetic is easy to follow and check. The method is what matters.

5.1 Step 1: Define the Problem

- **Business goal:** reduce monthly churn by contacting likely leavers with a retention offer.
- **ML task:** *binary classification*: will the customer leave in the next 30 days? (1 = churn, 0 = stays).
- **Success measure:** not accuracy alone, but **recall of churners** balanced against the **cost of wasted offers**.
- **Constraint:** the marketing team wants to understand *why* a customer is flagged.

5.2 Step 2: Data

Item	Detail
Size	5000 customers, 12 features, one target (`churn`)
Features	Tenure (months), monthly bill, data usage, number of support calls, contract type (monthly / yearly), payment method, city
Class balance	20% churn, 80% stay: **imbalanced**
Problems found in EDA	3% missing values in `data_usage`; one impossible tenure (-2); duplicate rows; `monthly_bill` is right-skewed

5.3 Step 3: Preprocessing

1. Remove duplicates; fix the invalid tenure; drop the customer ID (an identifier, not a feature).
2. **Split first:** 80% training (4000), 20% test (1000), *stratified* so both keep 20% churn (so the test set has 200 churners and 800 stayers).
3. Impute missing `data_usage` with the **median** (skewed), *learned from training data only*; add a missing-indicator column.
4. One-hot encode contract type, payment method and city; log-transform the skewed bill.
5. Standardise numeric features (needed for KNN and logistic regression).
6. All steps live inside a **pipeline** so cross-validation stays leak-free.

5.4 Step 4: Baseline

Predict "stays" for everyone. Test-style result: accuracy = 80%, but recall = 0%, precision undefined, and every churner is missed. Any useful model must beat this on *recall/F1*, not just

accuracy. This is the accuracy paradox in action.

5.5 Step 5: Model Selection with Cross-Validation

Three candidate classifiers from this course are compared with **stratified 5-fold cross-validation on the training set** using F_1 of the churn class.

Model	CV mean F_1	Std over folds	Comment
Baseline (majority)	0.00	0.00	Reference point
Naïve Bayes	0.57	0.03	Fast; independence assumption violated (bill and usage correlated)
KNN ($k = 7$)	0.55	0.04	Slower at prediction; sensitive to scaling
Logistic regression	**0.62**	**0.03**	Best and stable; coefficients are explainable

Hyperparameters: KNN's k and logistic regression's regularisation `C` were tuned by grid search *inside* the cross-validation. **Decision:** choose **logistic regression**, as it scores highest, is stable across folds and satisfies the explainability requirement.

5.6 Step 6: Check for Overfitting and Underfitting

Model	Training F_1	Validation F_1	Diagnosis
Logistic regression	0.64	0.62	Small gap, moderate score: well-fitted (slight bias, little variance)
KNN, $k = 1$	1.00	0.48	**Overfitting**: memorises training data
KNN, $k = 7$	0.66	0.55	Mild overfitting, much better than $k = 1$
Logistic regression with very strong regularisation	0.35	0.34	**Underfitting**: both scores low

The chosen model shows a small train/validation gap and similar scores, so neither major problem is present. If the score were too low, we would add features (for instance *support calls in the last 30 days*) rather than more data.

5.7 Step 7: Final Evaluation on the Test Set (Used Once)

The chosen model is retrained on all 4000 training rows and evaluated on the 1000 test customers (200 churners, 800 stayers).

Worked Example: Confusion matrix at threshold 0.5

	Pred. Churn	Pred. Stay	Total
Actual Churn	TP = 120	FN = 80	200
Actual Stay	FP = 60	TN = 740	800
Total	180	820	1000

$$\text{Accuracy} = \frac{120+740}{1000} = 0.86, \qquad \text{Precision} = \frac{120}{180} = 0.667,$$
$$\text{Recall} = \frac{120}{200} = 0.60, \qquad F_1 = \frac{2 \cdot 120}{2 \cdot 120 + 60 + 80} = \frac{240}{380} = 0.632.$$

The model beats the baseline's 80% accuracy, and the test F_1 (0.63) matches the cross-validation estimate (0.62), **confirming that the model generalises**. But 80 of the 200 churners are still missed.

5.8 Step 8: Choosing the Threshold from Business Costs

Suppose each **missed churner (FN)** costs the company **Rs. 5000** in lost margin, and each **false alarm (FP)** costs **Rs. 500** for a wasted retention offer. Lowering the decision threshold from 0.5 to 0.3 flags more customers:

Setting	TP	FP	FN	Precision	Recall	Expected cost
No model (baseline)	0	0	200	—	0.00	Rs. 10,00,000
Threshold 0.5	120	60	80	0.667	0.60	Rs. 4,30,000
Threshold 0.3	**150**	**120**	**50**	**0.556**	**0.75**	**Rs. 3,10,000**

Cost $= 5000 \times FN + 500 \times FP$:

- Baseline: $5000 \times 200 = 10{,}00{,}000$.
- Threshold 0.5: $5000 \times 80 + 500 \times 60 = 4{,}00{,}000 + 30{,}000 = 4{,}30{,}000$.
- Threshold 0.3: $5000 \times 50 + 500 \times 120 = 2{,}50{,}000 + 60{,}000 = 3{,}10{,}000$.

At threshold 0.3 accuracy drops (from 0.86 to $\frac{150+680}{1000} = 0.83$) and precision falls, yet the **business cost is lower by Rs. 1,20,000** per 1000 customers. F_1 at threshold 0.3 is $\frac{300}{300+120+50} = 0.638$. **Lesson: choose the threshold by cost, not by accuracy.**

5.9 Step 9: Deployment and Monitoring

1. Deploy as a nightly batch job scoring all active customers; flag those above threshold 0.3.
2. Run a small **A/B test**: contact flagged customers in one group, none in a control group, and compare actual churn.
3. **Monitor** feature distributions, the share of customers flagged, and real churn; watch for drift (new plans, competitor offers).
4. **Retrain** quarterly or when performance falls. Review fairness (are some city or age groups flagged unfairly?) and protect customer data.

5.10 What the Case Study Demonstrated

Concept	Where it appeared
Problem framing	Binary classification driven by a business goal
Preprocessing and leakage control	Split first; imputers and scalers fitted on training data inside a pipeline
Baseline	Majority-class predictor: 80% accuracy but 0% recall
Cross-validation	Stratified 5-fold F_1 to compare Naïve Bayes, KNN, logistic regression
Overfitting / underfitting	$k = 1$ KNN overfit; over-regularised model underfit
Confusion matrix and metrics	Accuracy, precision, recall and F_1 on the test set
Threshold and costs	Threshold 0.3 saved Rs. 1,20,000 despite lower accuracy
Deployment	A/B testing, drift monitoring, retraining, fairness

5.11 Code Skeleton for the Whole Case

```
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          stratify=y, random_state=42)
num = ["tenure", "monthly_bill", "data_usage", "support_calls"]
cat = ["contract", "payment", "city"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median", add_indicator=True)),
                      ("sc", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])

pipe = Pipeline([("prep", prep),
                 ("clf", LogisticRegression(max_iter=1000, class_weight="balanced"))])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(pipe, {"clf__C": [0.01, 0.1, 1, 10]}, cv=cv, scoring="f1")
grid.fit(X_tr, y_tr)                                    # model selection by CV

proba = grid.predict_proba(X_te)[:, 1]
for t in (0.5, 0.3):                                    # business-driven threshold
    y_hat = (proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_te, y_hat).ravel()
    print(t, "cost =", 5000 * fn + 500 * fp)
print(classification_report(y_te, (proba >= 0.3).astype(int)))
```

Exam Tip

Case-study questions ask you to *apply* the pipeline to a scenario (churn, loan default, spam, disease screening). Use this skeleton as your answer: (1) define task and metric, (2) data and preprocessing without leakage, (3) baseline, (4) candidate models with cross-validation, (5) check over/underfitting, (6) confusion matrix and chosen metrics on the test set, (7) deployment and monitoring. Always justify *why* the chosen metric fits the problem.

Pro Corner (for professionals)

Making the case study production-grade. Use **calibrated** probabilities (isotonic or Platt scaling) so the threshold derived from costs, $t^* = \frac{c_{FP}}{c_{FP}+c_{FN}} = \frac{500}{500+5000} \approx 0.09$ for a well-calibrated model, is meaningful; in the case study the practical optimum differs because the probabilities are not perfectly calibrated and an offer also has a success probability. Replace the fixed cost model with **uplift modelling** (predict who is *persuadable*, not merely who will churn), evaluate with **time-based validation** (train on earlier months, test on later), and track **population stability index** for drift.

6 Revision Toolkit

6.1 One-Page Summary

Topic	Remember this
Model validation	Estimate performance on unseen data. Train fits, validation tunes, test judges *once*. Always compare with a baseline; avoid leakage and test-set reuse.
Generalisation gap	Validation (or test) error − training error. Large gap ⇒ overfitting.
Underfitting	Too simple; high bias; train and validation errors both high. Fix: richer model, better features, less regularisation.
Overfitting	Too complex; high variance; train error tiny, validation error large. Fix: more data, simpler model, regularisation, early stopping, pruning, cross-validation.
Bias–variance	Error = Bias^2 + Variance + Noise. Complexity ↑ gives bias ↓, variance ↑.
Cross-validation	k-fold: train on $k-1$ folds, validate on one, rotate, average. Default $k=5$ or 10; stratify for classification; time-split for time series.
Applications	Healthcare, finance, retail, transport, agriculture, NLP, vision, security. Name task, learning type, algorithm, metric.
Deployment	Monitor drift, retrain, A/B test, check fairness and privacy.
Case study	Define goal → preprocess (no leakage) → baseline → CV model comparison → over/underfit check → test metrics → cost-based threshold → monitor.

6.2 Formula Sheet

Name	Formula
Generalisation gap	$E_{\text{val}} - E_{\text{train}}$
Bias–variance	$\mathbb{E}[(y-\hat{f})^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$
Ridge loss	$J = \text{MSE} + \lambda \sum w_j^2$
Lasso loss	$J = \text{MSE} + \lambda \sum \lvert w_j \rvert$
k**-fold CV score**	$\bar{s} = \frac{1}{k}\sum_{i=1}^{k} s_i$
Standard deviation across folds	$\sqrt{\frac{\sum (s_i - \bar{s})^2}{k-1}}$
Accuracy CI (95%)	$\hat{p} \pm 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$
Accuracy, precision, recall	$\frac{TP+TN}{N}$, $\frac{TP}{TP+FP}$, $\frac{TP}{TP+FN}$
F_1	$\frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}$
Expected cost	$c_{FP} \cdot FP + c_{FN} \cdot FN$

6.3 Commonly Confused Pairs

Pair	First	Second
Validation vs test set	Used repeatedly to tune and choose	Used once for the final unbiased score
Overfitting vs underfitting	Memorises noise; large gap	Too simple; both errors high
Bias vs variance	Systematic error from simple assumptions	Sensitivity to the training sample
Hold-out vs cross-validation	One split; fast but noisy	Many splits; reliable but k times the cost
k-fold vs stratified k-fold	Random folds	Folds keep class proportions
Parameter vs hyperparameter	Learned from data	Chosen before training
Data drift vs concept drift	Input distribution changes	Input–output relationship changes
Offline vs online evaluation	On stored data	On live users (A/B test)

6.4 Short-Answer Practice

1. **What is model validation?** Estimating a model's performance on unseen data, using data not used for fitting, to compare models and tune settings.
2. **Why do we need a separate test set?** Validation data influences our choices, so only an untouched test set gives an unbiased final estimate.
3. **What is a baseline model?** A trivial predictor (majority class or mean) used as a yardstick.
4. **Define overfitting.** The model fits training data, including noise, too closely and performs poorly on new data.
5. **Define underfitting.** The model is too simple to capture the pattern and performs poorly on both training and new data.
6. **Give three ways to reduce overfitting.** More data, regularisation, early stopping (also pruning, dropout, simpler model, cross-validation).
7. **Give two ways to reduce underfitting.** More expressive model; better features (or less regularisation).
8. **What does a learning curve show?** Training and validation error against training-set size, revealing bias or variance problems.
9. **What is k-fold cross-validation?** Splitting data into k folds, training on $k-1$ and validating on one in turn, then averaging the k scores.
10. **Why is stratification used?** To keep class proportions equal in every fold or split, especially with imbalanced classes.
11. **What is LOOCV?** Cross-validation with $k = n$: each sample is the validation set once.
12. **Why can cross-validation leak data?** If preprocessing is fitted on all data before splitting; use a pipeline.
13. **Name two real-world ML applications and their learning type.** Spam filtering (supervised classification); customer segmentation (unsupervised clustering).
14. **What is data drift?** A change in the distribution of input data after deployment, which can degrade accuracy.
15. **Why choose a threshold using costs?** Different errors have different real costs; the best

threshold minimises expected total cost, not error count.

6.5 Long-Answer Outlines

Question	Skeleton that earns full marks
Explain model validation techniques	Why validation; hold-out three-way split with diagram; baseline; metrics; hyperparameter tuning; cross-validation; pitfalls (leakage, test reuse); golden rules.
Explain overfitting and underfitting	Definitions; three-panel diagram; comparison table; bias–variance; learning curves; remedies for each; numerical example (polynomial degrees or early stopping).
Explain cross-validation	Motivation; k-fold steps; 5×5 diagram; numerical mean$\pm$std; variants (stratified, LOOCV, time-series); use for tuning; leakage; pros and cons.
Discuss applications of ML	Five or more domains with task, learning type, algorithm, metric; deployment and drift; ethics and limitations.
Case study: churn / loan / spam prediction	Problem and metric; data and preprocessing; baseline; models compared by CV; over/underfitting check; confusion matrix and metrics; threshold by cost; deployment and monitoring.

6.6 Multiple-Choice Practice

1. The set used only once at the very end is the: (a) training set (b) validation set (c) test set (d) bootstrap set
2. High training accuracy and low test accuracy indicates: (a) underfitting (b) overfitting (c) good fit (d) bias
3. Low accuracy on both training and test data indicates: (a) underfitting (b) overfitting (c) leakage (d) variance
4. Overfitting is associated with: (a) high bias (b) high variance (c) zero noise (d) low complexity
5. Which helps reduce overfitting? (a) more complex model (b) regularisation (c) removing validation data (d) training longer without checks
6. Early stopping uses: (a) training loss only (b) validation loss (c) test loss (d) none
7. In 5-fold CV with 100 samples each round trains on: (a) 20 (b) 50 (c) 80 (d) 100
8. A model has fold scores $0.80, 0.90, 0.85$. The CV mean is: (a) 0.80 (b) 0.85 (c) 0.90 (d) 0.87
9. Stratified k-fold preserves: (a) feature scale (b) class proportions (c) time order (d) sample IDs
10. For time-ordered data use: (a) random shuffle k-fold (b) time-series split (c) LOOCV (d) bootstrap only
11. LOOCV on n samples trains the model: (a) once (b) k times (c) n times (d) n^2 times
12. A baseline classifier for imbalanced data is often: (a) majority class (b) random forest (c) deep network (d) clustering
13. Fitting a scaler on the entire dataset before cross-validation causes: (a) underfitting (b) data leakage (c) faster training (d) regularisation
14. Hyperparameters are tuned using: (a) test data (b) validation data or CV (c) future data (d) no data
15. Which is NOT an application of unsupervised learning? (a) customer segmentation (b) anomaly detection (c) spam classification with labels (d) topic discovery

16. A change in the input data distribution after deployment is: (a) data drift (b) overfitting (c) regularisation (d) stratification
17. Expected cost with $FN = 50$, $FP = 120$, $c_{FN} = 5000$, $c_{FP} = 500$ is: (a) 2,50,000 (b) 3,10,000 (c) 4,30,000 (d) 60,000
18. A larger test set generally gives a(n): (a) wider confidence interval (b) tighter confidence interval (c) biased model (d) overfit model

Tip

Answers: 1-c, 2-b, 3-a, 4-b, 5-b, 6-b, 7-c, 8-b, 9-b, 10-b, 11-c, 12-a, 13-b, 14-b, 15-c, 16-a, 17-b, 18-b.

6.7 Glossary

Term	Meaning
Baseline	Simple reference model for comparison
Bias	Error from overly simple assumptions
Variance	Error from sensitivity to the training sample
Capacity / complexity	Flexibility of a model to fit patterns
Cross-validation	Rotating train/validation splits to estimate performance
Data drift	Change in input distribution over time
Early stopping	Halting training when validation error starts to rise
Fold	One of the k parts in k-fold CV
Generalisation	Performing well on unseen data
Grid search	Exhaustive search over hyperparameter combinations
Hold-out	A single split of data into train and test (or validation)
Hyperparameter	Setting chosen before training
Learning curve	Error versus training-set size
LOOCV	Leave-one-out cross-validation ($k = n$)
MLOps	Practices for deploying, monitoring and maintaining ML systems
Nested CV	CV inside CV to tune and evaluate without bias
Pruning	Removing tree branches that fit noise
Regularisation	Penalty that discourages overly complex models
Stratification	Keeping class proportions in each split
Test set	Final untouched data for the unbiased score
Validation set	Data used to tune and choose models

End of Module 5 — Model Evaluation and Applications
Introduction to Machine Learning • Course instructor: Prof. Rajesh Nag